SOFTWARE TASK 2

Submitted By : JANVI AMBALIYA

Employee ID : 39172

Date : March 2, 2026

Topic : SHA256 The Final Result

Step 1: Random Seed Generation

This step generates a unique, 2-byte random number to act as a "salt" or "seed" for the subsequent calculations. It reads raw data from /dev/urandom, converts it into an unsigned decimal format using the od command, and strips any extra spaces. This ensures that every time the script runs, the final hash will be different even if the system hardware remains the same.

```
jambaliya@vvdn-Latitude-3420: ~/practice/training/shell
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ ^C
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ SEED=$(head -c 2 /dev/urandom | od -An -tu2 | tr -d ' ')
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ echo "Step 1: Random Seed is -> $SEED"
Step 1: Random Seed is -> 31094
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$
```

Step 2: File Descriptor Multiplied by Seed

In this stage, the script identifies the current count of open File Descriptors (FD) for the process. This count is then multiplied by the random seed generated in the first step to produce a large integer. This step introduces process-specific variability into the final data string.

```
jambaliya@vvdn-Latitude-3420: ~/practice/training/shell
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ FD_COUNT=$(ls /proc/self/fd | wc -l)
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ PART1=$(($FD_COUNT * $SEED))
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ echo "Step 2: FD ($FD_COUNT) * Seed ($SEED) = $PART1"
Step 2: FD (4) * Seed (31094) = 124376
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$
```

Step 3: MAC Address Extraction and Reversal

The script retrieves the hardware MAC address of the primary network interface from the system path /sys/class/net/. It then extracts the last three octets (the final six characters), removes the colon separators, and reverses the string. This adds a layer of hardware-based identification to the raw data.

```
jambaliya@vvdn-Latitude-3420: ~/practice/training/shell
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ MAC=$(cat /sys/class/net/$(ls /sys/class/net | head -n 1)/address)
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ PART2=$(echo $MAC | cut -d: -f4-6 | tr -d ':' | rev)
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ echo "Step 3: MAC ($MAC) -> Last 3 Reversed = $PART2"
Step 3: MAC (00:be:43:51:2c:94) -> Last 3 Reversed = 49c215
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$
```

Step 4: SSH Directory Inode XOR Operation

This step retrieves the unique Inode number of the ~/.ssh directory using the stat command. A bitwise XOR operation is then performed between this Inode number and the initial random seed. This provides a unique file-system identifier that is masked by the seed for security.

```
jambaliya@vvdn-Latitude-3420: ~/practice/training/shell
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ INODE=$(stat -c %i ~/.ssh)
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ PART3=$((INODE ^ SEED))
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ echo "Step 4: Inode ($INODE) XOR Seed = $PART3"
Step 4: Inode (16127122) XOR Seed = 16170125
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$
```

Step 5: Final Raw Concatenation

The three previously generated components—the FD product, the reversed MAC octets, and the XORed Inode—are joined together into a single "raw" string. This concatenation creates a unique, complex data string that represents the combined state of the hardware, file system, and random seed.

```
jambaliya@vvdn-Latitude-3420: ~/practice/training/shell
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ RAW_RESULT="${PART1}${PART2}${PART3}"
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ HASH_RESULT=$(echo -n "$RAW_RESULT" | sha256sum | awk '{print $1}')
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ echo "FINAL RAW CONCATENATION: $RAW_RESULT"
FINAL RAW CONCATENATION: 49c21516170125
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$
```

Step 6: SHA256 Hash Generation

In the final step, the concatenated raw string is piped into the sha256sum utility. This cryptographic algorithm processes the input and produces a 64-character hexadecimal hash. This result serves as a secure, fixed-length digital fingerprint of the entire multi-step process.

```
jambaliya@vvdn-Latitude-3420: ~/practice/training/shell
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$ echo "FINAL SHA256 HASH: $HASH_RESULT"
FINAL SHA256 HASH: 98936a1f3d1201c40be5f8166714e8dadf7ed89cd1b7f65c3d3d05e75b25480c
jambaliya@vvdn-Latitude-3420:~/practice/training/shell$
```